

# Reward as Observation: Learning Reward-Based Policies for Rapid Adaptation

Morgan Byrd<sup>1\*</sup>, Maks Sorokin<sup>1</sup>, Robert Wright<sup>2</sup>, and Sehoon Ha<sup>1</sup>

**Abstract**—This paper explores a reward-based policy to achieve zero-shot transfer between source and target environments with completely different observation spaces. While humans can demonstrate impressive adaptation capabilities, deep neural network policies often struggle to adapt to a new environment and require a considerable amount of samples for successful transfer. Instead, we propose a novel reward-based policy only conditioned on rewards and actions, enabling zero-shot adaptation to new environments with completely different observations. We discuss the challenges and feasibility of a reward-based policy and then propose a practical algorithm for training. We demonstrate that a reward policy can be trained within three different environments, Pointmass, Cartpole, and 2D Car Racing, and transferred to completely different observations, such as different color palettes or 3D rendering, or Stretch robot navigation in Habitat-Sim, in a zero-shot manner. We also demonstrate that a reward-based policy can further guide the training of an observation-based policy in the target environment.

## I. INTRODUCTION

Humans readily adapt when the appearance of a familiar task changes. A gamer who masters a racing game can play immediately after a graphics engine upgrade, because the underlying rules and controls remain the same. Similarly, a robot trained to navigate a corridor should not need to relearn from scratch when its camera is replaced with a higher-resolution, wider Field-of-View or upgraded from RGB to RGB-D. In both cases, the core task structure, including dynamics, controls, and reward, is preserved; only the observation representation changes. However, typical deep RL policies treat such observation shifts as entirely new problems, requiring full retraining and discarding all previously learned task knowledge, because standard policies are tightly coupled to the specific observation space used during training.

We consider the problem of transferring a policy between two Markov Decision Process (MDP) environments that share the same underlying task structure, including state space, action space, transition dynamics, and reward function, but differ entirely in their observation space. Most prior transfer work in robotics focuses on transition dynamics shift, as in sim-to-real transfer [1], [2], [3], [4]. Instead, we address the complementary case: fixed dynamics, but a completely different observation space. We impose no constraint on the relationship between source and target observations; they may differ arbitrarily in modality, dimensionality, or structure. We intentionally fix the dynamics to isolate the observation shift

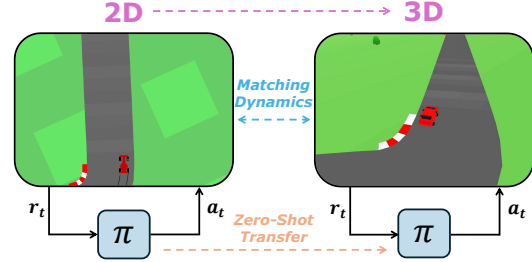


Fig. 1. A reward-based policy enables zero-shot transfer from the 2D Car Racing environment to the 3D Car Racing environment by taking reward and action histories as the only input.

problem, and we expect our approach to compose naturally with existing methods that handle transition shifts.

Our key insight is that the reward signal directly encodes the task structure and remains consistent across both MDPs, and therefore serves as a natural bridge to enable rapid transfer between environments with different observations. Let us consider a driving task: if the reward stays high, the agent should continue its current behavior; if the reward drops, a steering correction is needed. This intuition holds whether the agent observes a 2D top-down view, a 3D camera feed, or the onboard camera of a Stretch mobile robot in a scanned 3D environment. This suggests that a scalar reward signal alone may carry sufficient information to address the challenge of drastic observation shift, but only if it is sufficiently dense and informative. Under this condition, the reward signal has the potential to provide effective guidance in the target environment, regardless of how the observations change.

We propose a reward-based policy as a transferable teacher that bridges source and target environments under drastic observation shifts. Training such a policy is inherently challenging, as the reward signal is only a scalar projection of the underlying state space with limited information for control. To overcome this, we adopt an LSTM-based architecture that processes the temporal history of reward and action pairs, and incorporate a guidance loss from an observation-based expert policy to stabilize training. Once trained in the source environment, the reward-based policy is deployed in the target environment as a teacher, and we use DAgger to train an observation-based policy significantly faster than learning from scratch.

We demonstrate that a reward-based policy can serve as an effective transferable teacher across environments with arbitrary observation shifts. First, reward-based policies achieve 66% to 95% of state-based performance across three environments. Second, they transfer zero-shot under drastic observation shifts, including 2D-to-3D rendering

<sup>1</sup>Georgia Institute of Technology, Atlanta, GA, 30308, USA

<sup>2</sup>Georgia Tech Research Institute, Atlanta, GA, 30308, USA

\*Correspondence to abyrd45@gatech.edu

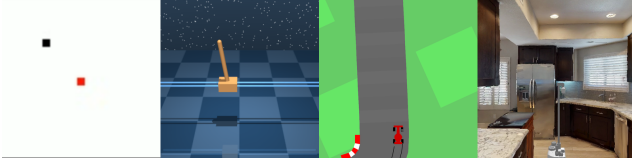


Fig. 2. Evaluation environments: Pointmass, Cartpole, Car Racing, and a Habitat-Sim environment for transfer testing.

changes and deployment on photorealistic Habitat-Sim [5] reconstructions of real building interiors with a Stretch mobile robot. Third, using the reward-based policy as a DAgger teacher enables substantially faster learning in the target environment. Finally, reward estimation proves far more tractable than state estimation under limited data.

The technical contributions of this paper are:

- We define the reward-based MDP and discuss its theoretical properties, including feasibility conditions and an optimality gap bound (Appendix).
- We propose a practical implementation for training reward-based policies, and a DAgger-based algorithm that uses the reward-based policy as a zero-shot teacher to train an observation-based policy in the target environment.
- We demonstrate reward-based policy training across diverse environments and validate rapid transfer under drastic observation shifts, including 2D-to-3D transfer and zero-shot deployment on a Stretch mobile robot in photorealistic Habitat-Sim reconstructions of real building interiors.

## II. RELATED WORK

**Transfer via Expert Demonstrations.** Transfer learning in RL leverages existing information for rapid adaptation in a new scenario, which has been approached by a wide range of algorithms. One possible approach is learning from demonstration [6], where expert demonstrations are available to guide the learning process. Offline methods can use these demonstrations for offline RL [7], [8], [9] and for pretraining to learn value functions or policies [10], [11]. Online methods use expert demonstrations to improve policy exploration during learning, and include policy iterations [12], [13], policy gradients [14], [15], and Q-learning approaches [16], [17]. Our reward-based policy serves a similar role, as it automatically provides guidance in the target environment through the shared reward signal.

**Transfer via Representation Learning.** Another way to transfer knowledge is representation learning, which establishes reusable representations in the action or state spaces that can be shared between source and target environments. Progressive networks [18] and PathNet [19] directly train common reusable representations and leverage them during policy training. There exists a line of works that split networks into general and reusable modules [20], [21], [22] and learn an invariant feature space in order to better transfer between tasks. [23], [24] learn inter-task mappings to deal with state differences between tasks. Successor representations [25] seek to separate the environment dynamics from the reward

function. These have seen widespread use [26], [27], [28] for transferring knowledge for different tasks with the same state and action spaces. [29] does a similar approach of separating dynamics and reward for learning a better policy for transfer. In contrast, our method avoids the observation alignment problem by using the shared reward signal as a bridge.

**Transfer via Policy Distillation.** The most relevant method of transfer for our work is to transfer at the policy level. A popular method to transfer involves distilling information from an expert policy. This method uses supervised learning to match action distributions between the student and teacher policies, and is widely used for robotic applications operating under observation shift between student and teacher [1], [2], [3], [4]. Policies can also be directly reused for updating policies in a transfer setting, as in [30], [31], [32]. Our method is most closely related to this line of work, but uses a reward-based policy trained in the source environment as the teacher, requiring no observation alignment.

**Reward-Conditioned Policies.** Another very similar sounding work to ours is the reward-conditioned policy [33]. However, this work trains standard observation-based policies with an additional reward conditioning that allows for using suboptimal trajectories as optimal supervision. In contrast, our method uses the reward signal to train a transferable teacher in the source environment, which accelerates learning in the target environment.

## III. PROBLEM DEFINITION: TRANSFER ACROSS DIFFERENT OBSERVATION SPACES

A Partially Observable Markov decision process (POMDP) is a popular tool for modeling a sequential decision problem. It is defined as a tuple  $(S, O, A, R, T)$ , where  $S$  is the state space,  $O$  is the observation space,  $A$  is the action space,  $T$  is the transition function that defines how states change given an action, and  $R$  is the reward function. The policy  $\pi : O \mapsto A$  takes action based on the given observation without knowing the underlying state  $s \in S$ , and we want to find an optimal policy that maximizes the discounted cumulative sum of the reward over time:  $\sum_{t=0}^T \gamma^t R(s_t, a_t)$ .

Our scenario defines the transfer problem as training a reusable policy from a source to a target environment. We assume the source and target environments should have the same transition, reward, action, and state but may have very different observation spaces. The source environment is defined as  $\mathcal{M}_{\text{src}} = (S, O_{\text{src}}, A, R, T)$  and the target environment as  $\mathcal{M}_{\text{tgt}} = (S, O_{\text{tgt}}, A, R, T)$ , where we impose no constraint on the relationship between  $O_{\text{src}}$  and  $O_{\text{tgt}}$ . In general, a policy trained in  $\mathcal{M}_{\text{src}}$  cannot be directly reused in  $\mathcal{M}_{\text{tgt}}$ , as the input dimensionality and structure of  $O_{\text{src}}$  and  $O_{\text{tgt}}$  may differ arbitrarily. Without specific assumptions on the relationship between the two observation spaces, the policy must be retrained from scratch in  $\mathcal{M}_{\text{tgt}}$ , which makes transfer challenging.

TABLE I

MAXIMUM REWARD (% OF OPTIMAL) FOR POINTMASS AS A FUNCTION OF THE NUMBER OF DIMENSIONS OF STATE SPACE.

| Dimension | State-based | Reward-based |
|-----------|-------------|--------------|
| 1         | 99.83       | 98.73        |
| 2         | 99.60       | 97.11        |
| 3         | 99.48       | 95.94        |
| 4         | 99.34       | 88.12        |
| 5         | 99.14       | 85.96        |
| 6         | 99.05       | 57.18        |

#### IV. REWARD-BASED POLICY

##### A. Reward-based MDP

We define a reward-based MDP  $\mathcal{M}_E^{\text{rwd}} = (S, O^{\text{rwd}}, A, R, T)$  of an original environment  $E$  by replacing the original observation space  $O$  with a reward-based observation  $O^{\text{rwd}} = \{(r_{t-k}, a_{t-k})\}_{k=0}^{K-1}$ , which is a history of the most recent  $K$  reward and action pairs. Here,  $K$  is a hyperparameter controlling the length of the history. The corresponding reward-based policy  $\pi^{\text{rwd}} : O^{\text{rwd}} \mapsto A$  takes only the reward/action history as input, with no access to the original observations. This formulation requires a dense reward signal; without sufficient reward coverage over the state space, the policy cannot acquire enough information for effective control.

The key property of the reward-based MDP is that it is independent of the original observation space  $O$ . Since both  $\mathcal{M}_{\text{src}}$  and  $\mathcal{M}_{\text{tgt}}$  share the same reward and action spaces,  $\mathcal{M}_{\text{src}}^{\text{rwd}}$  and  $\mathcal{M}_{\text{tgt}}^{\text{rwd}}$  are identical. A policy trained in  $\mathcal{M}_{\text{src}}^{\text{rwd}}$  can therefore be directly applied in  $\mathcal{M}_{\text{tgt}}^{\text{rwd}}$  without any modification. In contrast, a standard observation-based policy trained in  $\mathcal{M}_{\text{src}}$  cannot be reused in  $\mathcal{M}_{\text{tgt}}$ , as  $O_{\text{src}}$  and  $O_{\text{tgt}}$  may differ arbitrarily in dimensionality and structure.

##### B. Properties

1) *Difficulty: Limited Observability:* Learning a policy from only reward information is more difficult than using a standard observation-based approach. This is because the reward is a scalar projection of the, in general,  $D$ -dimensional state space that governs the dynamics of the task. The observation representation for a policy is crucial [34], and this reduction in information makes the learning task much more challenging as the partial observability makes estimating the value and advantage functions more difficult. Exploration is also negatively impacted by the lower information feedback.

This difficulty is particularly pronounced in higher dimensions. The optimality gap of a reward-based policy is bounded linearly by the state reconstruction error  $\epsilon$  (see Appendix). Because inferring state from scalar reward signals implicitly requires orthogonal probing steps (analogous to trilateration), the reconstruction error is expected to scale as  $\mathcal{O}(c^D)$  for some  $c > 1$ . This exponential growth in  $\epsilon$  with respect to  $D$  offers a theoretical explanation for the sharp performance degradation observed as dimensionality increases.

2) *Feasibility: Low-dimensional Environments:* Despite these difficulties, reward-based policies remain feasible in some environments. Takens' Embedding Theorem [35] provides a theoretical basis: a  $D$ -dimensional state can in

principle be reconstructed from a scalar observation history of length  $K \geq 2D + 1$ , given a sufficiently smooth observation function. This supports the use of a reward-action history as a sufficient input for policy learning, and in practice we find that reward-based policies can be trained with off-the-shelf RL algorithms.

To illustrate this, consider a simple 1D Pointmass environment where the reward is defined as a negative distance to the origin. A reward-based agent can estimate its position by checking how the reward changes in response to its actions. Even in  $D$ -dimensional Pointmass environments, an analytical algorithm similar to trilateration can determine the location from a sequence of reward and action pairs. However, as the dimension increases, the time required grows nearly exponentially.

Table I illustrates the feasibility and the dimensional scaling of reward-based policies. State-based policies maintain near-optimal performance regardless of dimension. Reward-based policies remain feasible at least up to 6D, which confirms that learning from reward history alone is achievable. However, with a fixed history length  $K$ , performance degrades sharply as  $D$  increases, achieving only 57% of the maximum reward in the 6D environment. This is consistent with the  $\mathcal{O}(c^D)$  error bound: when  $K$  is fixed, the reconstructive capacity does not scale with  $D$ , and the optimality gap widens accordingly.

3) *Requirement: Dense Rewards:* Reward-based policy learning requires a dense reward function. Because the reward serves as the sole observation, it must provide an informative signal throughout the state space. A sparse reward, such as one given only at the goal, leaves the agent without any useful feedback for most of the trajectory, which makes learning infeasible in practice.

4) *Limitation: Suboptimal Exploration:* Reward-based policies also exhibit suboptimal exploration behavior. In a 2D Pointmass environment, an observation-based agent with direct access to  $x$  and  $y$  can act optimally from the start. A reward-based agent, however, can only infer its distance to the goal from a single reward, and must take at least two information-gathering actions before it can localize itself and act optimally. This overhead is inherent to the reward-based formulation, and is expected to worsen in higher-dimensional or more complex environments.

##### C. Training Reward-Based Policies

Based on the analysis above, we identify two key components for training a reward-based policy effectively:

- 1) Temporal history of reward/action pairs.
- 2) Guidance from an observation-based expert policy.

**Temporal History via LSTM.** While Takens' Embedding Theorem suggests a fixed history length  $K \geq 2D + 1$  is sufficient in principle, selecting the right  $K$  requires knowing  $D$  in advance, which varies across environments. Instead, we use an LSTM [36] to implicitly maintain a variable-length history, which avoids the need to manually tune  $K$ . Each reward/action pair is first processed by a small MLP for feature extraction, and the resulting features are passed to the LSTM. The LSTM output is then used as input to the

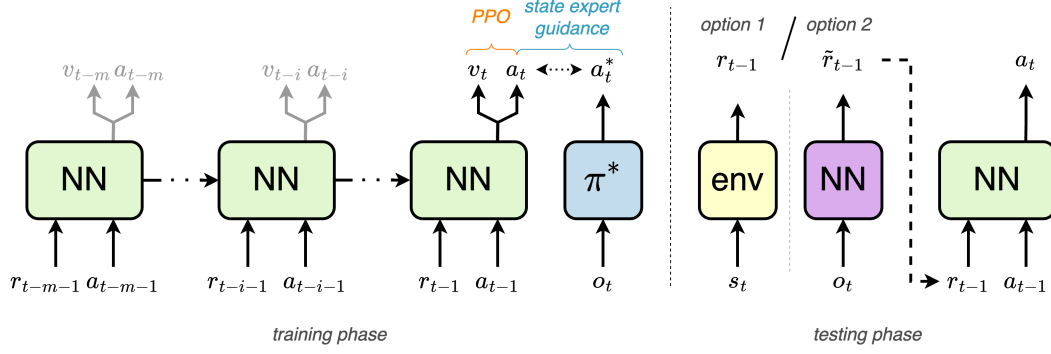


Fig. 3. Architecture. The reward-based policy  $\pi_R$  is trained using expert state-based guidance from  $\pi^*$  and standard training in RL with PPO.

policy and value networks, trained with PPO [37] using code from [38].

**Expert Guidance.** Temporal history alone is not sufficient for effective learning in practice. Without additional signal, the policy struggles to explore productively under the limited observability of reward-only inputs. To address this, we add a supervised guidance loss that regresses the policy actions toward those of a pre-trained observation-based expert  $\pi^*$ . The full training loss is  $L = L_{\text{PPO}} + L_{\text{guidance}}$ , where  $L_{\text{guidance}} = (a_t - a_t^*)^2$ .

#### D. Training Observation-Based Policies in Target Environments

Once a reward-based policy  $\pi^{\text{rd}}$  is trained in the source environment  $\mathcal{M}_{\text{src}}$ , it can be deployed directly in the target environment  $\mathcal{M}_{\text{tgt}}$ , since both share the same reward and action spaces. We leverage this transferability to train an observation-based policy  $\pi_{\text{tgt}}$  in  $\mathcal{M}_{\text{tgt}}$  using DAgger [39], where  $\pi^{\text{rd}}$  serves as the teacher. At each iteration,  $\pi_{\text{tgt}}$  collects trajectories in  $\mathcal{M}_{\text{tgt}}$ , and the reward-based policy provides action labels for the visited states. The resulting observation-based policy  $\pi_{\text{tgt}}$  operates purely on target observations at inference time, without requiring access to rewards.

## V. RESULTS

### A. Reward-Based Policies Train Successfully Across Diverse Environments

We evaluate on three source environments shown in Fig. 2, selected in order of increasing task complexity. The simplest environment tests the basic feasibility of reward-based learning, while the most complex evaluates transfer under a maximally challenging observation shift.

**Pointmass** is a 2D environment where the agent moves a point mass to a target location with kinematic actions  $s_{t+1} = s_t + a_t$ . The reward is the negative exponential of the distance to the goal, which provides a dense signal throughout the state space. This environment is intentionally simple and serves as a controlled testbed for reward-based policy learning.

**Cartpole** is the pole-balancing task from the DeepMind Control Suite [40]. The agent applies continuous torque to balance a pole on a moving cart. The reward is proportional to the cosine of the pole angle, which remains dense and

informative throughout the episode, as well as regularizing terms for torque, velocity, and positioning.

**Car Racing** uses the Gymnasium [41] 2D top-down environment as the source. The original reward is sparse, so we replace it with a dense formulation:  $r = r_{\text{distance}} \cdot r_{\text{angle}}$ , where  $r_{\text{distance}} = \exp(-d)$  penalizes distance to the next waypoint and  $r_{\text{angle}} = \cos(\theta)$  penalizes heading deviation from the track direction. To test the most extreme case of observation shift, we developed a custom 3D rendering engine that reproduces the same track layout and transition dynamics as the 2D source. This setup enables a direct 2D-to-3D transfer evaluation, where the observation space changes from a simple top-down view to a full 3D camera perspective, while all other aspects of the task remain identical.

Fig. 4 shows learning curves across three random seeds, with state-based policies as the performance upper bound. Reward-based policies achieve 95%, 66%, and 70% of state-based performance in Pointmass, Cartpole, and Car Racing, respectively. While reward-based policies do not fully reach the state-based upper bound, this gap is consistent with the theoretical bound in Theorem 1. Since the amount of information discarded by the scalar reward signal grows with state dimensionality, the achieved performance is respectable given the fundamental partial observability. A Pointmass agent navigates to the goal after a few exploratory steps. In Car Racing, the agent occasionally deviates from the track centerline to acquire additional reward signal before correcting course. This level of performance is sufficient for the transfer experiments in the following sections, as demonstrated in the supplementary video.

### B. Reward-Based Policies Transfer Zero-Shot Under Drastic Observation Shifts

Because a reward-based policy takes only reward and action as input, its behavior is independent of the observation representation. A policy trained in the source environment can be deployed in any target environment that shares the same reward and action spaces, without modification.

We first evaluate on a color palette swap, where the observation colors are changed while all other aspects of the environment remain the same. Fig. 5 shows example observation shifts across all three environments. Table II



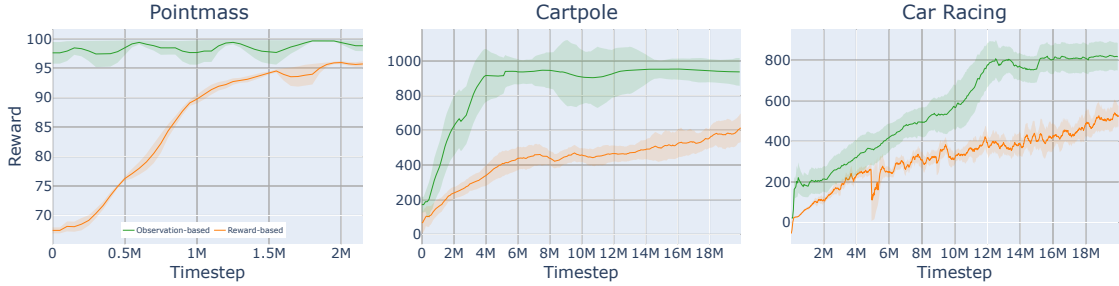


Fig. 4. Learning curves for reward-based and state-based policies across three environments. State-based policies serve as the performance upper bound.

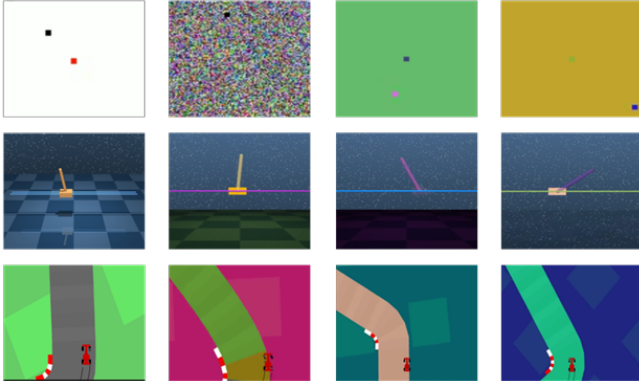


Fig. 5. A reward-based policy transfers to any observation variant in a zero-shot manner, including intentionally difficult color palette swaps.

TABLE II

ZERO-SHOT TRANSFER UNDER COLOR PALETTE SWAPS. MEAN REWARD VALUE  $\pm$  STANDARD DEVIATION OVER 50 TRIALS. PERFORMANCE IS FUNCTIONALLY IDENTICAL IN BOTH OBSERVATION DOMAINS.

| Task       | Original Observation | Shifted Observation |
|------------|----------------------|---------------------|
| Pointmass  | 97.06 $\pm$ 1.36     | 95.96 $\pm$ 2.47    |
| Cartpole   | 758.74 $\pm$ 62.79   | 755.45 $\pm$ 52.63  |
| Car Racing | 775.37 $\pm$ 225.27  | 715.93 $\pm$ 264.28 |

shows that reward-based policies achieve nearly identical performance in both the original and palette-swapped environments. This confirms that observation changes have no effect on reward-based policy behavior.

We then evaluate more challenging shifts: transferring from the 2D Gymnasium Car Racing environment to both a 3D rendering of the same task (Fig. 1) and to photorealistic Habitat-Sim reconstructions of real building interiors driven by a Stretch mobile robot (Fig. 6). For the 3D rendering, we match the transition function to the 2D source to isolate the observation shift. The reward-based policy transfers directly and demonstrates optimal driving behavior indistinguishable from the 2D expert reference. For Habitat, the same checkpoint is used with no retraining on the navmesh-derived corridor tracks recorded in two real HM3D [42] building scans, succeeding in both scenes and producing sustained navigation despite the entirely different rendering pipeline and robot embodiment. As in Car Racing, the reward  $r = r_{\text{distance}} \cdot r_{\text{angle}}$  is computed from the pose relative to the recorded waypoints, which we assume can be obtained via localization during real deployments. Neither transfer requires

retraining or observation alignment, showing that the reward signal bridges arbitrarily different observation spaces.

### C. Reward-Based Policies Accelerate Observation-Based Policy Learning in the Target Environment via DAgger

The zero-shot transfer results above establish that a reward-based policy operates in the target environment without modification. We now use this property to train an observation-based policy in the target environment, with the reward-based policy serving as a DAgger [39] teacher.

We use the 3D Car Racing environment as the target. Training an observation-based policy from scratch in this setting is challenging due to high-dimensional pixel observations. We use a CNN-LSTM architecture that first processes pixel input with a CNN and passes features to an LSTM before the actor and critic networks. As shown in Fig. 7, learning from scratch converges slowly; notably, this trend is visible even on a logarithmic timestep scale.

Using the reward-based policy as a DAgger teacher, the student policy learns substantially faster and reaches near optimal performance in 70,000 steps. Fig. 8 shows a motion strip comparing the source 2D environment and the target 3D environment, demonstrating that the student policy reproduces consistent driving behavior across both observation spaces.

One may question whether the upfront training cost in the source environment offsets the benefit. Two considerations argue otherwise. First, a reward-based policy transfers to any target environment that shares the same reward and action spaces, so the training cost is amortized across multiple transfers. Second, per-step simulation cost often differs substantially between environments: source environments are typically lightweight and support massively parallel rollouts, while target environments rendered by commercial graphics engines can be substantially more costly per step. Concentrating training in the inexpensive source and transferring rapidly to the expensive target therefore yields a clear practical advantage in such settings.

### D. Reward Estimation Succeeds Where State Estimation Fails

One may suggest a simpler alternative: learn a state estimator from target observations and apply the existing state-based policy directly. We compare this baseline against reward-based transfer using the DMC Cartpole task, where the target environment provides image observations.



Fig. 6. Motion strip of the reward-based policy deployed zero-shot (no retraining) in Habitat-Sim on two real building scans (HM3D), driving a Stretch mobile robot. Top: scene 00801 (bedroom-to-kitchen corridor). Bottom: scene 00807 (open living/dining area).

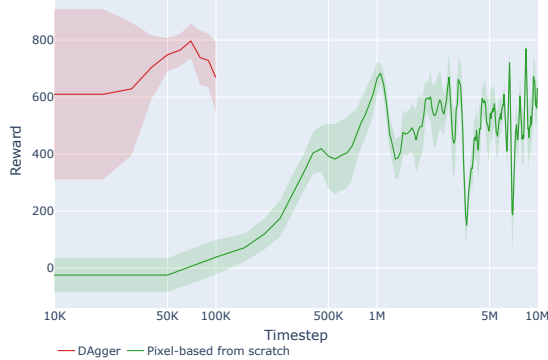


Fig. 7. Learning curves for the 3D Car Racing environment on a logarithmic timestep scale. Learning from scratch (CNN-LSTM) is compared against DAgger distillation using the reward-based policy as teacher.

TABLE III

A REWARD-BASED POLICY CAN BE USED EVEN WITHOUT A REWARD AT INFERENCE, AS ESTIMATING THE REWARD FROM AN IMAGE IS EASIER THAN THE FULL STATE, ALLOWING THE POLICY TO BE USED DIRECTLY FROM A SMALL SET OF TRAINING. MEANS AND STANDARD DEVIATIONS ARE REPORTED OVER 100 TRIALS WITH EACH POLICY.

| Policy       | Original Domain   | Image Domain       |
|--------------|-------------------|--------------------|
| State-based  | 915.49 $\pm$ 2.24 | 189.41 $\pm$ 12.54 |
| Reward-based | 836.49 $\pm$ 4.46 | 735.83 $\pm$ 65.48 |
| Random       | -                 | 192.78 $\pm$ 36.91 |

Both methods collect 100,000 samples from the target image domain and train their respective estimators using supervised learning. Table III shows the results. The state-based policy with a learned state estimator performs at the level of a random policy (189 vs. random baseline 192). The five-dimensional state is too difficult to recover accurately from limited data. The reward-based policy with a learned reward estimator maintains substantial performance (735), close to its source-domain score of 836.

This gap reflects a fundamental difference in estimation difficulty. A reward is a one-dimensional scalar and can be accurately recovered from limited data. A state is higher-dimensional and far more sensitive to estimation error under the same data budget. Reward-based transfer therefore reduces the transfer problem to estimating the simplest possible signal, while state-based transfer requires recovering full high-dimensional structure.

### E. Expert Guidance Improves Training in Complex Environments

We compare reward-based policies trained with and without expert guidance to validate this design choice. Fig. 9 shows that guidance provides a small but consistent improvement in Pointmass and Cartpole, while the benefit is substantially larger in Car Racing.

The difference across environments reflects the role of exploration under limited observability. In Pointmass and Cartpole, suboptimal exploration still yields task reward, so the policy can improve without guidance. In Car Racing, track deviation terminates the episode and eliminates the reward signal. Expert guidance prevents early episode termination during training. This allows the policy to observe sufficient reward signal for effective learning.

### VI. CONCLUSION

This paper proposes a reward-based policy framework for zero-shot transfer across environments with different observation spaces. We show that a policy conditioned only on reward and action history can be trained with an LSTM architecture and expert guidance, and can serve as a transferable teacher to accelerate observation-based policy learning in a target environment via DAgger, including photorealistic Habitat-Sim reconstructions of real building interiors with a Stretch mobile robot. We further show that reward estimation from target observations is substantially easier than state estimation, which provides an additional practical advantage when rewards are not directly available.

Several directions remain for future work. This work assumes identical transition functions between source and target environments to isolate the observation shift problem. Combining reward-based policies with domain randomization or system identification could extend the framework to settings where both observation and dynamics shift simultaneously.

Finally, this work evaluates reward-based policies in simulation, including photorealistic Habitat-Sim reconstructions of real building interiors; physical hardware deployment remains untested. It is also essential to examine physical observation space adaptations in real-world robotic deployments, such as transitioning mobile platforms from LiDAR to RGB-D sensors or equipping manipulators with extended Field-of-View (FoV) cameras. Prior work has demonstrated that real-world robot learning is feasible with careful experimental design [43], [44], and similar setups could be used to validate our framework.

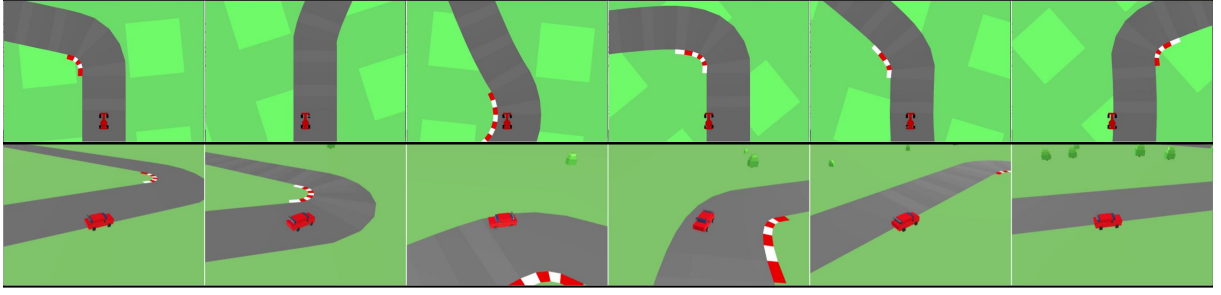


Fig. 8. Motion strip comparing the source 2D Car Racing environment (top) and the target 3D environment (bottom). The reward-based teacher enables the student policy to reproduce consistent driving behavior across both observation spaces.

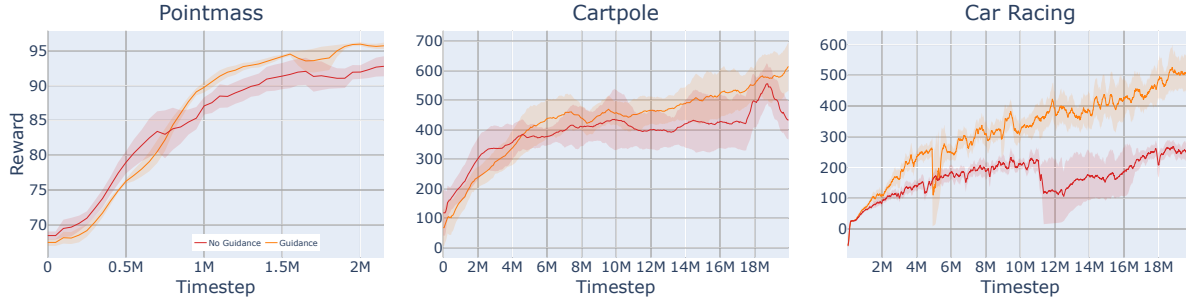


Fig. 9. Ablation on expert guidance across three environments. The benefit of guidance increases with task difficulty.

on physical hardware. In parallel, real-world rewards can often be estimated directly from sensor data without explicit instrumentation: for example, lane deviation in a driving task can be inferred from a camera image. Demonstrating the framework on a physical robotic platform, either with directly measured or estimated rewards, would substantially strengthen its impact for the robotics community.

## VII. ACKNOWLEDGMENTS

ChatGPT was used for generating the code for the 3D Car Racing environment and for minor editing. All content was reviewed and validated by the authors.

## REFERENCES

- [1] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “Rma: Rapid motor adaptation for legged robots,” 2021.
- [2] Y. Liang, K. Ellis, and J. Henriques, “Rapid motor adaptation for robotic manipulator arms,” *arXiv preprint arXiv:2312.04670*, 2024.
- [3] M. Byrd, J. Crandell, M. Das, J. Inman, R. Wright, and S. Ha, “Privileged-dreamer: Explicit imagination of privileged information for rapid adaptation of learned policies,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 3640–3646.
- [4] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022. [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abk2822>
- [5] X. Puig *et al.*, “Habitat 3.0: A co-habitat for humans, avatars and robots,” 2023.
- [6] S. Schaal, “Learning from demonstration,” in *Advances in Neural Information Processing Systems*, M. Mozer, M. Jordan, and T. Petsche, Eds., vol. 9. MIT Press, 1996. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/1996/file/68d13cf26c4b4f932e3eff990093ba-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/1996/file/68d13cf26c4b4f932e3eff990093ba-Paper.pdf)
- [7] Y. Ma, Y.-X. Wang, and B. Narayanaswamy, “Imitation-regularized offline learning,” 2019.
- [8] M. Yang and O. Nachum, “Representation matters: Offline pretraining for sequential decision making,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 11 784–11 794.
- [9] A. Li, B. Boots, and C.-A. Cheng, “Mahalo: Unifying offline reinforcement learning and imitation learning from observations,” in *International Conference on Machine Learning*. PMLR, 2023.
- [10] D. Silver *et al.*, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan 2016. [Online]. Available: <https://doi.org/10.1038/nature16961>
- [11] K. Stachowicz, A. Bhorkar, D. Shah, I. Kostikov, and S. Levine, “FastRLAP: A System for Learning High-Speed Driving via Deep RL and Autonomous Practicing,” 2023. [Online]. Available: <https://arxiv.org/abs/2304.09831>
- [12] B. Piot, M. Geist, and O. Pietquin, “Boosted bellman residual minimization handling expert demonstrations,” in *Machine Learning and Knowledge Discovery in Databases*, T. Calders, F. Esposito, E. Hüllermeier, and R. Meo, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 549–564.
- [13] J. Chemali and A. Lazaric, “Direct policy iteration with demonstrations,” in *Proceedings of the 24th International Conference on Artificial Intelligence*, ser. IJCAI’15. AAAI Press, 2015, p. 3380–3386.
- [14] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Overcoming exploration in reinforcement learning with demonstrations,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 6292–6299.
- [15] B. Kang, Z. Jie, and J. Feng, “Policy optimization with demonstrations,” 2018.
- [16] T. Brys, A. Harutyunyan, H. B. Suay, S. Chernova, M. E. Taylor, and A. Nowé, “Reinforcement learning from demonstration through shaping,” in *Proceedings of the 24th International Conference on Artificial Intelligence*, ser. IJCAI’15. AAAI Press, 2015, p. 3352–3358.
- [17] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband, G. Dulac-Arnold, J. Agapiou, J. Z. Leibo, and A. Gruslys, “Deep q-learning from demonstrations,” in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAI Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI’18/IAAI’18/EAAI’18. AAAI Press, 2018.
- [18] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, “Progressive neural networks,” 6 2016. [Online]. Available: <http://arxiv.org/abs/1606.04671>
- [19] C. Fernando, D. Banarse, C. Blundell, Y. Zwols, D. Ha, A. A. Rusu, A. Pritzel, and D. Wierstra, “Pathnet: Evolution channels gradient

descent in super neural networks,” 1 2017. [Online]. Available: <http://arxiv.org/abs/1701.08734>

[20] J. Andreas, D. Klein, and S. Levine, “Modular multitask reinforcement learning with policy sketches,” in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML’17. JMLR.org, 2017, p. 166–175.

[21] C. Devin, A. Gupta, T. Darrell, P. Abbeel, and S. Levine, “Learning modular neural network policies for multi-task and multi-robot transfer,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 2169–2176.

[22] A. Gupta, C. Devin, Y. Liu, P. Abbeel, and S. Levine, “Learning invariant feature spaces to transfer skills with reinforcement learning,” 3 2017. [Online]. Available: <http://arxiv.org/abs/1703.02949>

[23] H. Bou-Ammar and M. E. Taylor, “Reinforcement learning transfer via common subspaces,” in *Adaptive and Learning Agents - International Workshop, ALA 2011, Held at AAMAS 2011, Taipei, Taiwan, May 2, 2011, Revised Selected Papers*, ser. Lecture Notes in Computer Science, P. Vrancx, M. Knudson, and M. Grzes, Eds., vol. 7113. Springer, 2011, pp. 21–36. [Online]. Available: [https://doi.org/10.1007/978-3-642-28499-1\\_2](https://doi.org/10.1007/978-3-642-28499-1_2)

[24] M. E. Taylor, P. Stone, and Y. Liu, “Transfer learning via inter-task mappings for temporal difference learning,” *Journal of Machine Learning Research*, vol. 8, no. 1, pp. 2125–2167, 2007.

[25] P. Dayan, “Improving generalization for temporal difference learning: The successor representation,” *Neural Computation*, vol. 5, no. 4, pp. 613–624, 1993.

[26] A. Barreto, D. Borsa, J. Quan, T. Schaul, D. Silver, M. Hessel, D. Mankowitz, A. Zidek, and R. Munos, “Transfer in deep reinforcement learning using successor features and generalised policy improvement,” 2018.

[27] J. Zhang, J. T. Springenberg, J. Boedecker, and W. Burgard, “Deep reinforcement learning with successor features for navigation across similar environments,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 2371–2378.

[28] G. Konidaris and A. Barto, “Autonomous shaping: Knowledge transfer in reinforcement learning,” 2006.

[29] A. Zhang, H. Satija, and J. Pineau, “Decoupling dynamics and reward for transfer learning,” 4 2018. [Online]. Available: <http://arxiv.org/abs/1804.10689>

[30] A. Barreto, W. Dabney, R. Munos, J. J. Hunt, T. Schaul, H. V. Hasselt, and D. Silver, “Successor features for transfer in reinforcement learning,” 2017.

[31] F. Fernández and M. Veloso, “Probabilistic policy reuse in a reinforcement learning agent,” in *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS ’06. New York, NY, USA: Association for Computing Machinery, 2006, p. 720–727. [Online]. Available: <https://doi.org/10.1145/1160633.1160762>

[32] J. Rajendran, A. S. Lakshminarayanan, M. M. Khapra, P. Prasanna, and B. Ravindran, “Attend, adapt and transfer: Attentive deep architecture for adaptive transfer from multiple sources in the same domain,” in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. [Online]. Available: <https://openreview.net/forum?id=Sy6iJDqlx>

[33] A. Kumar, X. B. Peng, and S. Levine, “Reward-conditioned policies,” *arXiv preprint arXiv:1912.13465*, 2019.

[34] J. T. Kim and S. Ha, “Observation space matters: Benchmark and optimization algorithm,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021.

[35] F. Takens, “Detecting strange attractors in turbulence,” 1981. [Online]. Available: <https://www.semanticscholar.org/paper/4039236d89b682a2ab7aa9baaa4057707161d9be>

[36] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, p. 1735–1780, nov 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>

[37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.

[38] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. Araújo, “Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms,” *Journal of Machine Learning Research*, vol. 23, no. 274, pp. 1–18, 2022. [Online]. Available: <http://jmlr.org/papers/v23/21-1342.html>

[39] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,”

in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, vol. 15. Fort Lauderdale, FL, USA: PMLR, 11–13 Apr 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>

[40] Y. Tassa *et al.*, “Deepmind control suite,” 1 2018. [Online]. Available: <http://arxiv.org/abs/1801.00690>

[41] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.

[42] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. M. Turner, E. Undersander, W. Galuba, A. Westbury, A. X. Chang, M. Savva, Y. Zhao, and D. Batra, “Habitat-matterport 3d dataset (HM3D): 1000 large-scale 3d environments for embodied AI,” in *Thirty-fifth Conference on Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021.

[43] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, “QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation,” *ArXiv*, vol. abs/1806.10293, 2018.

[44] S. Ha, P. Xu, Z. Tan, S. Levine, and J. Tan, “Learning to Walk in the Real World with Minimal Human Effort,” *Conference on Robot Learning*, pp. 1110–1120, 2020.

## APPENDIX

We formally bound the optimality gap between observation-based and reward-based policies.

**Definition 1** ( $\epsilon$ -Reconstructibility). *The state space  $S$  is  $\epsilon$ -reconstructible from  $O^{rd}$  if there exists a mapping  $\phi : O^{rd} \mapsto S$  such that for any history  $h_t \in O^{rd}$ :*

$$\|s_t - \phi(h_t)\| \leq \epsilon \quad (1)$$

where  $\epsilon \geq 0$  is the maximum reconstruction error.

To relate reconstruction error to policy performance, we assume the value function varies smoothly with the state.

**Assumption 1** (Lipschitz Continuity). *The optimal value function  $V_{\mathcal{M}}^*$  is  $L$ -Lipschitz continuous: for any  $s_1, s_2 \in S$ ,*

$$|V_{\mathcal{M}}^*(s_1) - V_{\mathcal{M}}^*(s_2)| \leq L\|s_1 - s_2\| \quad (2)$$

Under these two conditions, the optimality gap of the reward-based policy is bounded linearly by  $\epsilon$ .

**Theorem 1** (Optimality Bound). *The performance gap between the optimal observation-based policy and the optimal reward-based policy satisfies:*

$$V_{\mathcal{M}}^*(s_t) - V_{\mathcal{M}^{rd}}^*(h_t) \leq \frac{L\epsilon}{1-\gamma} \quad (3)$$

where  $\gamma \in [0, 1)$  is the discount factor.

*Proof.* Define an induced policy  $\hat{\pi}(h_t) = \pi^*(\phi(h_t))$ , which acts as if the reconstructed state were the true state. By Assumption 1, the single-step value error is bounded by:

$$|V_{\mathcal{M}}^*(s_t) - V_{\mathcal{M}}^*(\phi(h_t))| \leq L\|s_t - \phi(h_t)\| \leq L\epsilon \quad (4)$$

Accumulating this error over the infinite horizon under  $\hat{\pi}$ , and using the fact that the optimal reward-based policy satisfies  $V_{\mathcal{M}^{rd}}^* \geq V_{\mathcal{M}^{rd}}^{\hat{\pi}}$ , yields:

$$V_{\mathcal{M}}^*(s_t) - V_{\mathcal{M}^{rd}}^*(h_t) \leq \sum_{k=0}^{\infty} \gamma^k L\epsilon = \frac{L\epsilon}{1-\gamma} \quad (5)$$



The remaining question is how  $\epsilon$  scales with  $D$ . Recovering a  $D$ -dimensional state from scalar rewards requires at least  $D + 1$  independent probing steps, and the probability of sufficiently orthogonal measurements decreases exponentially with  $D$ . We therefore expect  $\epsilon(D) = \mathcal{O}(c^D)$  for fixed  $K$ , predicting an  $\mathcal{O}(c^D)$  optimality gap consistent with Table I.